❄ SNOWFLAKE

Snowpipe · Streams · Tasks · UDFs · Semi-Structured Data · Micro-partitioning · Caching · RBAC · Data Masking · Data Sharing · Cost Optimization

SESSION 1 — Data Loading & Automation

1.1 Bulk Loading vs Continuous Loading

Aspect	Bulk Loading (COPY INTO)	Continuous Loading (Snowpipe)
Trigger	Manual or scheduled	Automatic — event-driven when files arrive
Latency	Minutes to hours (batch)	Near real-time (seconds to minutes)
Compute	Uses your Virtual Warehouse	Uses Snowflake-managed serverless compute
Cost	Warehouse credits (per second)	Per-file + per-byte compute costs
Best for	Large historical loads, nightly ETL	Streaming feeds, IoT, real-time pipelines
Control	Full control over timing	Hands-off, automatic

1.2 COPY INTO — Advanced Options

```
-- ON_ERROR: what to do when a row fails
COPY INTO employees FROM @my_stage/data.csv
  FILE_FORMAT = (TYPE='CSV' SKIP_HEADER=1)
  ON_ERROR = 'CONTINUE';      -- skip bad rows, continue loading
-- ON_ERROR = 'SKIP_FILE';    -- skip the entire file if any error
-- ON_ERROR = 'ABORT_STATEMENT'; -- stop everything (DEFAULT)

-- PURGE: delete files from stage after successful load
COPY INTO orders FROM @my_stage/
  FILE_FORMAT = (TYPE='CSV' SKIP_HEADER=1)
  PURGE = TRUE;

-- FORCE: reload files already loaded (ignores load history)
COPY INTO orders FROM @my_stage/orders.csv
  FILE_FORMAT = (TYPE='CSV' SKIP_HEADER=1)
  FORCE = TRUE;

-- PATTERN: load only matching files
COPY INTO orders FROM @my_stage/
  PATTERN = '.*orders_2024_01.*\.csv'
  FILE_FORMAT = (TYPE='CSV' SKIP_HEADER=1);

-- Check what files Snowflake has already loaded (load metadata kept 64 days):
SELECT * FROM TABLE(INFORMATION_SCHEMA.COPY_HISTORY(
  TABLE_NAME=>'ORDERS',
  START_TIME=> DATEADD(HOURS,-24,CURRENT_TIMESTAMP)));

-- VALIDATE function: check a past load for errors
SELECT * FROM TABLE(VALIDATE(orders, JOB_ID=>'<copy_job_id>'));
```

> **TIP** Snowflake remembers which files were loaded for 64 days. If you run COPY INTO again with the same file, it will be skipped automatically — unless you use FORCE=TRUE.

1.3 Snowpipe — Continuous / Auto-Ingest Loading

Snowpipe is Snowflake's serverless, event-driven data ingestion service. When a new file lands in your stage, Snowpipe automatically loads it — no manual trigger needed.

How Snowpipe works:

1. A file arrives in S3 / Azure Blob / GCS bucket
2. Cloud sends an event notification (S3 Event Notification / Azure Event Grid)
3. Snowflake receives the notification through an SQS queue
4. Snowpipe picks up the file and loads it using the pipe's COPY statement
5. You see data in your table within seconds to minutes

```
-- Step 1: Create a stage pointing to S3
CREATE OR REPLACE STAGE auto_ingest_stage
  URL = 's3://my-bucket/incoming/'
  STORAGE_INTEGRATION = s3_int;

-- Step 2: Create a PIPE (defines the COPY logic)
CREATE OR REPLACE PIPE orders_pipe
  AUTO_INGEST = TRUE          -- enables event-based trigger
  COMMENT = 'Auto-ingest orders from S3'
AS
COPY INTO orders (order_id, customer_id, order_date, amount, status)
FROM (
  SELECT $1, $2, $3::DATE, $4::NUMBER(10,2), $5
  FROM @auto_ingest_stage
)
FILE_FORMAT = (TYPE='CSV' SKIP_HEADER=1);

-- Step 3: Get the SQS ARN to configure S3 event notification:
SHOW PIPES;
-- Copy the 'notification_channel' value → paste into AWS S3 Event settings

-- Monitor Snowpipe status:
SELECT SYSTEM$PIPE_STATUS('orders_pipe');

-- Check load history for a pipe:
SELECT * FROM TABLE(INFORMATION_SCHEMA.PIPE_USAGE_HISTORY(
  DATE_RANGE_START => DATEADD(HOUR,-24,CURRENT_TIMESTAMP),
  PIPE_NAME => 'orders_pipe'));

-- Manually trigger pipe (REST API call - for testing):
-- POST https://<account>.snowflakecomputing.com/v1/data/pipes/<pipe_name>/insert-
Files

-- Pause and resume a pipe:
ALTER PIPE orders_pipe SET PIPE_EXECUTION_PAUSED = TRUE;
ALTER PIPE orders_pipe SET PIPE_EXECUTION_PAUSED = FALSE;
```

⛔ **FREE TIER** Snowpipe AUTO_INGEST requires an external stage (S3/Azure/GCS) and cloud event configuration. On the free trial you can CREATE the pipe and test REST API calls, but the auto-trigger requires a real cloud bucket. You can simulate loading by calling the pipe REST API manually.

1.4 Streams — Change Data Capture (CDC)

A Stream is a Snowflake object that tracks all changes (INSERT, UPDATE, DELETE) made to a table since the stream was last consumed. Think of it as a changelog or transaction log.

Real-world analogy:

Imagine a newspaper delivery tracker. The stream records every new paper delivered (INSERT), every address change (UPDATE), every cancellation (DELETE). When you read the tracker, it marks those as 'consumed' and starts fresh.

```
-- Create a stream on a table:
CREATE OR REPLACE STREAM orders_stream ON TABLE orders
  APPEND_ONLY = FALSE;    -- tracks INSERT + UPDATE + DELETE
-- APPEND_ONLY = TRUE     -- only tracks new INSERTs (faster, less overhead)

-- Check stream details:
SHOW STREAMS;
DESCRIBE STREAM orders_stream;

-- *** Simulate changes to the source table ***
INSERT INTO orders VALUES (106, 2, '2024-04-01', 5500.00, 'PENDING');
UPDATE orders SET status='COMPLETED' WHERE order_id=103;
DELETE FROM orders WHERE order_id=105;

-- Read the stream to see what changed:
SELECT * FROM orders_stream;
-- Stream adds these special system columns:
-- METADATA$ACTION    : 'INSERT' or 'DELETE'
-- METADATA$ISUPDATE : TRUE if it was an UPDATE (shows as DELETE+INSERT pair)
-- METADATA$ROW_ID    : unique row identifier

-- Separate INSERTs, UPDATEs, DELETEs:
SELECT *, 'NEW ROW'     FROM orders_stream WHERE METADATA$ACTION='INSERT' AND NOT
METADATA$ISUPDATE;
SELECT *, 'UPDATED ROW' FROM orders_stream WHERE METADATA$ACTION='INSERT' AND
METADATA$ISUPDATE;
SELECT *, 'DELETED ROW' FROM orders_stream WHERE METADATA$ACTION='DELETE' AND NOT
METADATA$ISUPDATE;

-- IMPORTANT: Stream is consumed (reset) when you SELECT from it inside a DML:
-- Just SELECT does NOT consume the stream.
-- INSERT INTO ... SELECT FROM stream DOES consume it.
```

⚠ **IMPORTANT** Streams have an OFFSET — they track changes from when the stream was created or last consumed. If no task/DML consumes the stream within the table's data retention period, the stream becomes STALE and must be recreated.

1.5 Tasks — Scheduled SQL Execution

A Task is a Snowflake object that runs a SQL statement (or stored procedure) on a schedule — like a cron job inside Snowflake.

```
-- Create a simple task (runs every 5 minutes):
CREATE OR REPLACE TASK refresh_analytics_task
  WAREHOUSE = training_wh
  SCHEDULE = '5 MINUTE'
  COMMENT = 'Refresh analytics table every 5 minutes'
AS
INSERT INTO analytics.orders_summary
  SELECT DATE_TRUNC('DAY', order_date) as day,
         COUNT(*) as total_orders,
         SUM(amount) as total_revenue
  FROM raw.orders
  GROUP BY 1;

-- Tasks start in SUSPENDED state — must resume them:
ALTER TASK refresh_analytics_task RESUME;

-- Suspend a task:
ALTER TASK refresh_analytics_task SUSPEND;

-- Run a task manually (for testing):
EXECUTE TASK refresh_analytics_task;

-- Check task run history:
SELECT * FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(
  TASK_NAME => 'REFRESH_ANALYTICS_TASK',
  SCHEDULED_TIME_RANGE_START => DATEADD(HOUR,-24,CURRENT_TIMESTAMP)))
ORDER BY SCHEDULED_TIME DESC;

-- View all tasks:
SHOW TASKS;
```

TIP Use CRON expressions for more precise scheduling: SCHEDULE = 'USING CRON 0 8 * * MON-FRI America/New_York' — runs every weekday at 8 AM Eastern.

FREE TIER Tasks work on free tier! They use serverless compute by default (no warehouse needed if you use SERVERLESS instead of a named warehouse). Serverless tasks are billed per second of compute.

1.6 DAGs of Tasks — Dependent & Chained Tasks

A DAG (Directed Acyclic Graph) is a set of tasks where each task can trigger the next one. This lets you build complex pipelines like: Extract → Transform → Load → Aggregate → Report.

Key rule: Only the ROOT task (the first one) can have a SCHEDULE. Child tasks are triggered by their parent completing.

```
-- ================================
-- BUILD A 4-STEP PIPELINE:
-- Extract → Transform → Load → Report
-- ================================
```

```
-- STEP 1: Root Task (has the schedule)
CREATE OR REPLACE TASK task_extract
  WAREHOUSE = training_wh
  SCHEDULE = '60 MINUTE'    -- runs every hour
AS
INSERT INTO raw.orders_extract
SELECT * FROM raw.orders_staging WHERE processed = FALSE;

-- STEP 2: Transform (triggered after extract completes)
CREATE OR REPLACE TASK task_transform
  WAREHOUSE = training_wh
  AFTER task_extract        -- <-- defines dependency
AS
INSERT INTO analytics.orders_clean
SELECT order_id,
       customer_id,
       TRY_TO_DATE(order_date) as order_date,
       TRY_TO_NUMBER(amount) as amount,
       UPPER(status) as status
FROM raw.orders_extract;

-- STEP 3: Load summary (after transform)
CREATE OR REPLACE TASK task_load_summary
  WAREHOUSE = training_wh
  AFTER task_transform
AS
MERGE INTO analytics.orders_summary tgt
USING (SELECT DATE_TRUNC('DAY',order_date) as day,
              COUNT(*) as cnt, SUM(amount) as rev
       FROM analytics.orders_clean GROUP BY 1) src
ON tgt.day = src.day
WHEN MATCHED THEN UPDATE SET tgt.cnt=src.cnt, tgt.rev=src.rev
WHEN NOT MATCHED THEN INSERT VALUES (src.day, src.cnt, src.rev);

-- STEP 4: Report (after summary loaded)
CREATE OR REPLACE TASK task_report
  WAREHOUSE = training_wh
  AFTER task_load_summary
AS
INSERT INTO analytics.daily_report
SELECT CURRENT_TIMESTAMP as run_time, day, cnt, rev
FROM analytics.orders_summary
WHERE day = CURRENT_DATE;

-- *** Resume ALL tasks in the DAG (children first, then root) ***
ALTER TASK task_report       RESUME;
ALTER TASK task_load_summary RESUME;
ALTER TASK task_transform    RESUME;
ALTER TASK task_extract      RESUME;  -- root last!

-- View the DAG structure:
SELECT * FROM TABLE(INFORMATION_SCHEMA.TASK_DEPENDENTS(
  TASK_NAME => 'TASK_EXTRACT', RECURSIVE => TRUE));
```

1.7 Streams + Tasks Together — Event-Driven Pipeline

The most powerful pattern in Snowflake automation: use a Stream to detect changes, and a Task to process them. The task can check if the stream HAS data before running — saving compute.

```
-- Create stream on source table:
CREATE OR REPLACE STREAM orders_cdc_stream ON TABLE raw.orders;

-- Create task that ONLY runs if stream has new data:
CREATE OR REPLACE TASK process_orders_changes
  WAREHOUSE = training_wh
  SCHEDULE = '1 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('orders_cdc_stream')  -- <-- skip if no changes
AS
BEGIN
  -- Process new inserts:
  INSERT INTO analytics.new_orders
  SELECT order_id, customer_id, order_date, amount, status
  FROM orders_cdc_stream
  WHERE METADATA$ACTION = 'INSERT' AND NOT METADATA$ISUPDATE;

  -- Process deletes:
  DELETE FROM analytics.new_orders tgt
  USING (SELECT order_id FROM orders_cdc_stream
         WHERE METADATA$ACTION='DELETE') src
  ON tgt.order_id = src.order_id;
END;

ALTER TASK process_orders_changes RESUME;

-- Test: insert into orders and watch the task pick it up:
INSERT INTO raw.orders VALUES (200, 3, '2024-05-01', 8800.00, 'PENDING');
-- Within 1 minute, check:
SELECT * FROM analytics.new_orders WHERE order_id = 200;
```

TIP SYSTEM$STREAM_HAS_DATA() is your best friend — it prevents tasks from wasting compute credits when there is nothing to process. Always use it!

Questions — Session 1

Q1	What is the difference between Bulk Loading and Continuous Loading (Snowpipe)?
Q2	What does ON_ERROR='CONTINUE' do in COPY INTO?
Q3	Explain how Snowpipe AUTO_INGEST works step by step.
Q4	What special metadata columns does a Stream add, and what do they mean?
Q5	When is a Stream considered 'stale'?
Q6	What is a DAG of Tasks? What is the rule about which task can have a schedule?
Q7	What does SYSTEM$STREAM_HAS_DATA() do and why is it important?
Q8	What is the difference between APPEND_ONLY=TRUE and APPEND_ONLY=FALSE in streams?
Q9	Does a simple SELECT from a stream consume it? What operation consumes a stream?

Q10	In a 3-task DAG (A→B→C), which task do you resume last and why?

SESSION 2 — UDFs, Stored Procedures & Scripting

2.1 User Defined Functions (UDFs)

A UDF lets you create your own custom SQL functions. Once created, you can call them just like built-in functions (UPPER, ROUND, etc.). UDFs take inputs and return a SINGLE value.

Supported languages: SQL, JavaScript, Python, Java, Scala.

```
-- ============================================
-- UDF Example 1: SQL UDF — Clean a phone number
-- ============================================
CREATE OR REPLACE FUNCTION clean_phone(raw_phone VARCHAR)
RETURNS VARCHAR
LANGUAGE SQL
AS
$$
  -- Remove all non-digit characters
  REGEXP_REPLACE(raw_phone, '[^0-9]', '')
$$;

-- Test it:
SELECT clean_phone('+91 (98765) 43210') AS cleaned;
-- Result: 919876543210

-- ============================================
-- UDF Example 2: JavaScript UDF — Categorize salary
-- ============================================
CREATE OR REPLACE FUNCTION salary_band(sal FLOAT)
RETURNS VARCHAR
LANGUAGE JAVASCRIPT
AS
$$
  if (SAL < 50000)  return 'Entry Level';
  if (SAL < 80000)  return 'Mid Level';
  if (SAL < 120000) return 'Senior Level';
  return 'Executive';
$$;

-- Use it in a query:
SELECT emp_id, first_name, salary, salary_band(salary) AS band
FROM employees;

-- ============================================
-- UDF Example 3: Calculate age from birth date
-- ============================================
CREATE OR REPLACE FUNCTION calculate_age(birth_date DATE)
RETURNS INT
LANGUAGE SQL
AS
$$
  FLOOR(DATEDIFF('day', birth_date, CURRENT_DATE) / 365.25)
$$;
```

```
SELECT first_name, calculate_age('1990-05-15'::DATE) AS age FROM employees LIMIT
3;
```

2.2 Table UDFs (UDTFs) — Return Multiple Rows

A Table UDF (UDTF) returns a TABLE (multiple rows) instead of a single value. Use it when you need a function that generates or expands data.

```
-- Table UDF: generate a date range
CREATE OR REPLACE FUNCTION date_range(start_dt DATE, end_dt DATE)
RETURNS TABLE (calendar_date DATE)
LANGUAGE SQL
AS
$$
  SELECT DATEADD('day', seq4(), start_dt)::DATE AS calendar_date
  FROM TABLE(GENERATOR(ROWCOUNT => DATEDIFF('day', start_dt, end_dt) + 1))
$$;

-- Generate all dates in January 2024:
SELECT * FROM TABLE(date_range('2024-01-01'::DATE, '2024-01-31'::DATE));

-- Use it in a join to fill gaps in date data:
SELECT d.calendar_date, COALESCE(o.total_orders, 0) AS orders
FROM TABLE(date_range('2024-01-01'::DATE, '2024-01-31'::DATE)) d
LEFT JOIN (
  SELECT order_date, COUNT(*) AS total_orders FROM orders GROUP BY 1) o
  ON d.calendar_date = o.order_date
ORDER BY d.calendar_date;

-- Table UDF: split a comma-separated string into rows
CREATE OR REPLACE FUNCTION split_to_rows(input VARCHAR, delimiter VARCHAR)
RETURNS TABLE (item VARCHAR)
LANGUAGE SQL
AS
$$
  SELECT TRIM(value) AS item
  FROM TABLE(SPLIT_TO_TABLE(input, delimiter))
$$;

SELECT * FROM TABLE(split_to_rows('Python,SQL,Snowflake,dbt', ','));
```

2.3 Stored Procedures

A Stored Procedure is a reusable block of code that can contain logic (IF/ELSE, loops, transactions, multiple SQL statements). Unlike UDFs, stored procedures:

- Can execute multiple statements
- Can commit/rollback transactions
- Can call other stored procedures
- Return a single value (or nothing)
- Run with CALLER'S rights or OWNER'S rights (security model)

```
-- Stored Procedure: Load and validate data
```

```
CREATE OR REPLACE PROCEDURE load_and_validate_orders()
RETURNS VARCHAR
LANGUAGE SQL
EXECUTE AS CALLER
AS
$$
DECLARE
  v_count INT;
  v_message VARCHAR;
BEGIN
  -- Step 1: Load from staging
  INSERT INTO orders
  SELECT order_id, customer_id, order_date::DATE,
         amount::NUMBER(10,2), UPPER(status)
  FROM orders_staging
  WHERE order_id NOT IN (SELECT order_id FROM orders);

  -- Step 2: Count what was loaded
  SELECT COUNT(*) INTO :v_count FROM orders
  WHERE order_date = CURRENT_DATE;

  -- Step 3: Return result
  v_message := 'Loaded ' || v_count || ' orders for today.';
  RETURN v_message;
END;
$$;

-- Call the procedure:
CALL load_and_validate_orders();

-- ============================================
-- Stored Procedure with parameters and loop:
-- ============================================
CREATE OR REPLACE PROCEDURE archive_old_orders(days_old INT)
RETURNS VARCHAR
LANGUAGE SQL
AS
$$
DECLARE
  v_rows_archived INT;
BEGIN
  -- Archive orders older than specified days
  INSERT INTO orders_archive
  SELECT * FROM orders
  WHERE order_date < DATEADD('day', -days_old, CURRENT_DATE);

  v_rows_archived := SQLROWCOUNT;

  DELETE FROM orders
  WHERE order_date < DATEADD('day', -days_old, CURRENT_DATE);
```

```
  RETURN 'Archived ' || v_rows_archived || ' orders older than ' || days_old || '
days.';
END;
$$;

CALL archive_old_orders(365);  -- archive orders older than 1 year
```

2.4 Sequences — Auto-Increment IDs

A Sequence generates unique sequential numbers. Use it when you need auto-generated IDs.

```
-- Create a sequence:
CREATE OR REPLACE SEQUENCE order_id_seq
  START = 1000
  INCREMENT = 1
  COMMENT = 'Sequence for order IDs';

-- Use in INSERT:
INSERT INTO orders (order_id, customer_id, order_date, amount, status)
VALUES (order_id_seq.NEXTVAL, 1, CURRENT_DATE, 1200.00, 'PENDING');

-- Check current value:
SELECT order_id_seq.CURRVAL;

-- Use in CREATE TABLE (auto-assign IDs):
CREATE TABLE products (
  product_id INT DEFAULT product_seq.NEXTVAL,
  product_name VARCHAR(200),
  price NUMBER(10,2)
);
```

NOTE Unlike AUTOINCREMENT, sequences in Snowflake are not guaranteed to be gapless — they can have gaps due to multi-node execution. Use them for uniqueness, not for sequential counting.

2.5 Sampling — Working with Data Subsets

Sampling lets you query a random subset of rows — useful for exploration, testing, or ML dataset creation.

```
-- ============================================
-- METHOD 1: BERNOULLI (row-based, precise %)
-- ============================================
-- Each row has a 10% probability of being included:
SELECT * FROM orders SAMPLE BERNOULLI(10);  -- approx 10% of rows
-- Or use TABLESAMPLE synonym:
SELECT * FROM orders TABLESAMPLE BERNOULLI(10);

-- ============================================
-- METHOD 2: SYSTEM / BLOCK (block-based, faster)
-- ============================================
-- Selects random micro-partitions (faster for large tables):
SELECT * FROM orders SAMPLE SYSTEM(10);    -- approx 10% of blocks

-- ============================================
-- Fixed-size sampling (exact number of rows):
```

```
-- ============================================
SELECT * FROM orders SAMPLE (100 ROWS);    -- exactly 100 random rows

-- Repeatable sample (same seed = same rows every time):
SELECT * FROM orders SAMPLE BERNOULLI(10) SEED(42);

-- Create a training dataset (70% train, 30% test):
CREATE TABLE ml_train AS
SELECT * FROM orders SAMPLE BERNOULLI(70) SEED(100);

CREATE TABLE ml_test AS
SELECT * FROM orders
WHERE order_id NOT IN (SELECT order_id FROM ml_train);
```

Method	Based On	Speed	Precision	Use Case
BERNOULLI	Row-level probability	Slower	High — exact %	When row-level randomness matters (ML, analytics)
SYSTEM	Block/partition-level	Faster	Approx %	Quick exploration of large tables
Fixed ROWS	Exact count	Varies	Exact count	When you need exactly N rows

2.6 Estimated / Approximate Functions

For very large datasets, exact aggregations (COUNT DISTINCT) can be slow and expensive. Snowflake offers approximate functions that are extremely fast with ~2% error margin.

```
-- Exact COUNT DISTINCT (slow on billions of rows):
SELECT COUNT(DISTINCT customer_id) FROM orders;   -- exact but slow

-- Approximate COUNT DISTINCT (HyperLogLog algorithm):
SELECT APPROX_COUNT_DISTINCT(customer_id) FROM orders;  -- fast, ~2% error

-- Approximate percentiles:
SELECT APPROX_PERCENTILE(amount, 0.50) AS median_order,
       APPROX_PERCENTILE(amount, 0.90) AS p90_order,
       APPROX_PERCENTILE(amount, 0.99) AS p99_order
FROM orders;

-- MINHASH (similarity between datasets):
SELECT MINHASH(100, email) FROM customers;

-- T-Digest (distribution estimation):
SELECT TDIGEST_PERCENTILE(
  TDIGEST_AGG(amount, 100), 0.95) AS p95_amount
FROM orders;
```

Questions — Session 2

Q1	What is the difference between a UDF and a Stored Procedure?

Q2	What is a Table UDF (UDTF) and when would you use it instead of a regular UDF?
Q3	What languages can you write Snowflake UDFs in?
Q4	What does EXECUTE AS CALLER vs EXECUTE AS OWNER mean in stored procedures?
Q5	What is a Sequence and what is a limitation of Snowflake sequences?
Q6	What is the difference between BERNOULLI and SYSTEM sampling?
Q7	How do you get a repeatable (deterministic) sample in Snowflake?
Q8	What is APPROX_COUNT_DISTINCT and why would you use it?
Q9	Can a stored procedure call another stored procedure?
Q10	What SQL statement do you use to execute a stored procedure?

SESSION 3 — Semi-Structured & Unstructured Data

3.1 Semi-Structured Data in Snowflake

Semi-structured data is data that does NOT fit neatly into rows and columns — like JSON, XML, Avro, Parquet. Snowflake stores it natively in VARIANT columns and lets you query it with SQL.

Setup — Create table with JSON data:

```
CREATE OR REPLACE TABLE customer_events (
  event_id    NUMBER,
  event_time  TIMESTAMP_NTZ,
  event_raw   VARIANT   -- stores JSON
);

INSERT INTO customer_events VALUES
(1, '2024-01-10 09:00:00',
 PARSE_JSON('{
   "user_id": 101,
   "action": "purchase",
   "product": {"name": "Laptop", "category": "Electronics", "price": 75000},
   "tags": ["sale", "premium", "electronics"],
   "payment": {"method": "UPI", "bank": "HDFC"}
 }')),
(2, '2024-01-10 10:30:00',
 PARSE_JSON('{
   "user_id": 102,
   "action": "view",
   "product": {"name": "Mouse", "category": "Peripherals", "price": 1500},
   "tags": ["budget"],
   "payment": null
 }'));
```

3.2 Dot Notation & Bracket Notation

You can navigate inside a VARIANT column using dot notation (like JavaScript object access) or bracket notation (for dynamic keys or keys with special characters).

```
-- ============================================
-- DOT NOTATION: access nested fields
-- ============================================
SELECT
  event_raw:user_id::INT          AS user_id,
  event_raw:action::VARCHAR       AS action,
  event_raw:product.name::VARCHAR AS product_name,
  event_raw:product.price::FLOAT  AS price,
  event_raw:payment.method::VARCHAR AS payment_method
FROM customer_events;

-- ============================================
-- BRACKET NOTATION: use for dynamic keys
-- ============================================
SELECT
  event_raw['user_id']::INT             AS user_id,
  event_raw['product']['name']::VARCHAR AS product_name
```

```
FROM customer_events;

-- Access array elements by index (0-based):
SELECT
  event_raw:tags[0]::VARCHAR AS first_tag,
  event_raw:tags[1]::VARCHAR AS second_tag,
  ARRAY_SIZE(event_raw:tags)  AS tag_count
FROM customer_events;

-- ============================================
-- CASTING (::) is important — VARIANT returns VARIANT type
-- ============================================
-- Without cast: event_raw:price returns VARIANT
-- With cast:    event_raw:price::FLOAT returns FLOAT
-- Always cast when using in calculations or comparisons!

-- Filter on nested field:
SELECT * FROM customer_events
WHERE event_raw:product.category::VARCHAR = 'Electronics';

-- Null-safe access (returns NULL if path missing, no error):
SELECT event_raw:payment.method::VARCHAR AS method
FROM customer_events;  -- returns NULL for event 2 where payment is null
```

3.3 FLATTEN — Explode Arrays into Rows

FLATTEN is used to convert a JSON array into multiple rows — one row per array element. This is essential when you need to analyze array data at the element level.

```
-- ============================================
-- FLATTEN: convert tags array into rows
-- ============================================
SELECT
  ce.event_id,
  ce.event_raw:user_id::INT AS user_id,
  f.value::VARCHAR          AS tag
FROM customer_events ce,
  LATERAL FLATTEN(INPUT => ce.event_raw:tags) f;

-- Result:
-- event_id | user_id | tag
-- 1        | 101     | sale
-- 1        | 101     | premium
-- 1        | 101     | electronics
-- 2        | 102     | budget

-- FLATTEN output columns:
SELECT f.seq,     -- sequence number (row ordering)
       f.key,     -- key if object, null if array
       f.path,    -- path to this element
       f.index,   -- array index (0-based)
       f.value,   -- the actual value
       f.this     -- the parent element
```

```
FROM customer_events ce,
  LATERAL FLATTEN(INPUT => ce.event_raw:tags) f;

-- Flatten nested objects (expand all key-value pairs):
SELECT f.key, f.value::VARCHAR
FROM customer_events ce,
  LATERAL FLATTEN(INPUT => ce.event_raw:product) f
WHERE ce.event_id = 1;

-- Count events per tag:
SELECT f.value::VARCHAR AS tag, COUNT(*) AS events
FROM customer_events ce,
  LATERAL FLATTEN(INPUT => ce.event_raw:tags) f
GROUP BY tag
ORDER BY events DESC;

-- FLATTEN with OUTER => TRUE (keep rows even if array is empty/null):
SELECT ce.event_id, f.value::VARCHAR AS tag
FROM customer_events ce,
  LATERAL FLATTEN(INPUT => ce.event_raw:tags, OUTER => TRUE) f;
```

TIP Remember: LATERAL FLATTEN is used when the array is inside a column of the same table. The LATERAL keyword means 'for each row of the left table, apply this function'.

3.4 Unstructured Data in Snowflake

Beyond semi-structured data (JSON/XML), Snowflake can also work with truly unstructured files like PDFs, images, audio, video, and binary files stored in stages.

Snowflake provides 3 types of URLs to access unstructured files:

1. Scoped URL — Temporary access via a Snowflake-managed token

```
-- Scoped URLs are time-limited (default 24 hours)
-- Used for safe, expiring sharing of stage files
SELECT BUILD_SCOPED_FILE_URL(@my_stage, '/reports/sales_q1.pdf') AS scoped_url;

-- The URL looks like:
-- https://sfc-xx.snowflakecomputing.com/api/files/.../reports/sales_q1.pdf?to-
ken=...
-- Expires after the token TTL
```

2. File URL — Permanent URL (for authorized Snowflake users only)

```
-- File URLs are permanent but require Snowflake authentication
-- Only users with stage access can use them
SELECT BUILD_STAGE_FILE_URL(@my_stage, '/reports/sales_q1.pdf') AS file_url;

-- Good for: embedding in internal apps where users are already authenticated
```

3. Pre-signed URL — Public temporary URL (no auth required)

```
-- Pre-signed URLs are public — anyone with the URL can access the file
-- Expires after specified seconds
SELECT GET_PRESIGNED_URL(@my_stage, '/reports/sales_q1.pdf', 3600) AS pre-
signed_url;
-- 3600 = expires in 1 hour
```

```
-- Good for: sharing files with external users without Snowflake access
-- WARNING: treat like a password — anyone who has this URL can access the file!

-- Store file metadata in a directory table:
CREATE STAGE media_stage DIRECTORY = (ENABLE = TRUE);

-- Refresh directory table to pick up new files:
ALTER STAGE media_stage REFRESH;

-- List all files with metadata:
SELECT * FROM DIRECTORY(@media_stage);
-- Shows: name, size, last_modified, etag, file_url, relative_path
```

URL Type	Auth Required?	Expiry	Use Case
Scoped URL	Yes (Snowflake token)	Configurable (default 24h)	Internal temporary sharing within Snowflake
File (Stage) URL	Yes (Snowflake login)	Never expires	Internal apps with Snowflake-authenticated users
Pre-signed URL	No (public)	Set by you (seconds)	Sharing with external users/systems without Snowflake access

Questions — Session 3

Q1	What data type in Snowflake stores JSON? What is the maximum size?
Q2	What is the difference between dot notation and bracket notation when querying VARIANT data?
Q3	Why is casting (::) important when querying VARIANT columns?
Q4	What does LATERAL FLATTEN do? Give an example use case.
Q5	What is the OUTER => TRUE option in FLATTEN used for?
Q6	What are the 3 types of URLs for unstructured files? How do they differ?
Q7	When would you use a Pre-signed URL vs a Scoped URL?
Q8	What is a Directory Table in Snowflake?
Q9	How do you access the 2nd element of a JSON array stored in a VARIANT column?
Q10	You have a column 'details' with JSON: {"tags":["a","b","c"]}. Write SQL to get each tag as a separate row.

SESSION 4 — Performance & Optimization (2:20–3:00)

4.1 Micro-Partitioning — How Snowflake Stores Data

When you load data into Snowflake, it is automatically divided into MICRO-PARTITIONS — small, immutable, compressed files stored in the storage layer. You never create or manage these manually.

Key facts about micro-partitions:

- Each micro-partition is typically 50–500 MB of compressed data (before compression)
- Automatically created during data loading
- Stored in columnar format (like Parquet) — each column stored separately
- Snowflake maintains metadata about each partition: min/max values, null counts, distinct values per column
- Immutable: once written, a micro-partition is never modified — changes create NEW partitions

Query Pruning — Skip Partitions You Don't Need

Because Snowflake knows the min/max values of each column in every micro-partition, it can SKIP entire partitions that cannot possibly contain matching rows. This is called pruning.

```
-- Example: orders table has 1 billion rows across 10,000 micro-partitions
-- Query: find orders from January 2024
SELECT * FROM orders WHERE order_date BETWEEN '2024-01-01' AND '2024-01-31';

-- Snowflake checks metadata: which partitions have order_date values in Jan 2024?
-- If only 50 partitions contain January data → scans only those 50 (99.5%
pruned!)

-- Check pruning in query profile:
-- 'Partitions Scanned' vs 'Partitions Total' — the difference is pruned!

-- *** PRUNING WORKS BEST WHEN YOU FILTER ON COLUMNS THAT DATA IS SORTED BY ***
-- If data is loaded in date order (common for time-series), date filters prune
well.
-- Random order data = poor pruning.

-- View partition information for a table:
SELECT * FROM TABLE(INFORMATION_SCHEMA.TABLE_STORAGE_METRICS('ORDERS'));
```

4.2 Clustering Keys — Optimize Query Pruning

If your data is NOT naturally sorted on the column you filter by most, query pruning is poor. A Clustering Key tells Snowflake to reorganize (re-cluster) the data so that rows with similar values of the clustering column are in the same micro-partitions.

```
-- Without clustering: orders are stored in load order (random dates)
-- 'WHERE order_date = today' must scan most partitions

-- Add a clustering key on order_date:
ALTER TABLE orders CLUSTER BY (order_date);

-- Snowflake automatically maintains clustering in the background
-- (a process called automatic clustering - incurs credit charges)
```

```
-- Check clustering information:
SELECT SYSTEM$CLUSTERING_INFORMATION('orders', '(order_date)');
-- Returns: average_overlaps, average_depth, partition_count

-- Cluster on multiple columns:
ALTER TABLE orders CLUSTER BY (order_date, status);

-- Remove clustering key:
ALTER TABLE orders DROP CLUSTERING KEY;

-- Manually trigger reclustering (usually not needed):
ALTER TABLE orders RECLUSTER;
```

Clustering Depth, Overlap & Mapping

These metrics tell you how well-clustered a table is:

Metric	Meaning	Good Value
Overlap	How many partitions contain overlapping ranges of the clustering key	Low (close to 0) = well clustered
Depth	Average number of partitions a single clustering key value appears in	Close to 1 = well clustered
Constant	% of table that is optimally clustered	Close to 100% = well clustered

> **FREE TIER** Automatic Clustering consumes credits on the free tier. Be mindful — add clustering keys only to large production tables where query performance benefit justifies the cost. On free trial, you can set a clustering key but avoid letting it run long.

4.3 Caching in Snowflake — 3 Types

Snowflake has 3 levels of caching that can dramatically speed up queries and reduce compute costs.

Cache Type 1: Result Cache (Query Result Cache)

- Stores the EXACT result set of every query run
- If the same query is run again (with no underlying data changes), the result is returned instantly from cache — NO warehouse needed
- Cache lasts 24 hours (resets on warehouse restart or if data changes)
- FREE — no compute credits consumed when result cache is used

```
-- Run this query:
SELECT COUNT(*) FROM orders WHERE status='COMPLETED';
-- Takes 2 seconds on first run

-- Run EXACT same query again within 24 hours:
SELECT COUNT(*) FROM orders WHERE status='COMPLETED';
-- Returns instantly from result cache! 0 compute credits used.

-- Check if result cache was used: look at Query Profile
-- 'Query Result Reuse' = YES means result cache was used
```

```
-- Disable result cache for a session (for testing):
ALTER SESSION SET USE_CACHED_RESULT = FALSE;
```

Cache Type 2: Metadata Cache (Cloud Services Cache)

- Snowflake's Cloud Services layer caches table metadata, statistics, partition info
- Certain queries run entirely in the Cloud Services layer with NO warehouse at all
- Examples: COUNT(*) on a table, MIN/MAX values of a column, SHOW commands

```
-- These often run from metadata cache (no warehouse needed):
SELECT COUNT(*) FROM orders;          -- row count from metadata
SELECT MIN(order_date) FROM orders;   -- min from partition metadata
SELECT MAX(order_date) FROM orders;   -- max from partition metadata
SHOW TABLES;
SHOW DATABASES;

-- If query uses only metadata: 'Bytes scanned = 0' in query details
```

Cache Type 3: Warehouse Cache (Local Disk Cache)

- Each warehouse node has a local SSD cache of recently scanned data
- If you run a query, the data pages are cached on the warehouse nodes
- Subsequent queries that access the same data are faster (read from SSD, not storage)
- This cache is LOST when warehouse is suspended

```
-- First query: reads from cloud storage (slow)
SELECT * FROM large_orders_table WHERE order_date = '2024-01-15';

-- Second query (different filter, same table): faster (data cached on nodes)
SELECT * FROM large_orders_table WHERE order_date = '2024-01-16';

-- If warehouse is suspended between queries: cache is lost!
-- Tip: for analytical workloads, keep warehouse running to benefit from cache
```

Cache Type	What's Cached	Duration	Credits Used?	How to Identify
Result Cache	Exact query result set	24 hours	0 (FREE)	Query Profile: 'Query Result Reuse'
Metadata Cache	Table stats, partition metadata	Always (Cloud Svcs)	0 (FREE)	Bytes Scanned = 0
Warehouse Cache	Recently scanned data pages	Until WH suspended	Yes (WH must be running)	Lower 'Bytes Scanned from Remote' vs baseline

4.4 Query Profile — Analyzing Query Performance

Query Profile is Snowflake's built-in query explain/analyze tool. It shows you exactly what happened during a query execution.

How to access: Run a query → Click the query ID → Click 'Query Profile' tab

Key sections to look at:

- Overview: Total time, bytes scanned, rows produced, partitions scanned vs total
- Operator Tree: Visual breakdown of execution steps (TableScan, Filter, Aggregate, Join, Sort)
- Most Expensive Node: Snowflake highlights the slowest step
- Statistics Panel: For each node — rows input/output, bytes, time

```
-- After running a query, get its profile via SQL:
SELECT * FROM TABLE(GET_QUERY_OPERATOR_STATS('<query_id>'));

-- Common performance issues visible in Query Profile:

-- 1. Low pruning: 'Partitions Scanned' close to 'Partitions Total'
--    → Add clustering key or filter on clustered column

-- 2. Data Spilling (see below)

-- 3. Large joins: 'Bytes sent over network' is high
--    → Rearrange join order, filter data before joining

-- 4. UNION ALL producing too many rows
--    → Check if UNION is needed or can be replaced

-- Check query history:
SELECT query_id, query_text, execution_time, bytes_scanned,
       partitions_scanned, partitions_total
FROM TABLE(INFORMATION_SCHEMA.QUERY_HISTORY(
  DATE_RANGE_START => DATEADD(HOUR,-24,CURRENT_TIMESTAMP)))
ORDER BY execution_time DESC
LIMIT 20;
```

4.5 Data Spilling

Data spilling occurs when a query operation (sort, join, aggregation) requires MORE memory than the warehouse nodes have available. Snowflake then spills data to:

6. Local SSD disk (faster spill — slower than RAM but fast)
7. Remote cloud storage (slowest — significant performance impact)

How to detect in Query Profile: Look for 'Bytes spilled to local storage' and 'Bytes spilled to remote storage'. Remote spill > 0 is a red flag.

Solutions:

- Scale UP the warehouse (more RAM per node)
- Reduce data before joining: filter early, select only needed columns
- Break a large query into smaller steps with intermediate tables
- Avoid SELECT * — select only the columns you need

4.6 Resource Monitors — Cost Control

Resource Monitors let you set credit usage limits on warehouses and receive alerts or automatically suspend them when limits are hit.

```
-- Create a resource monitor:
```

```
CREATE OR REPLACE RESOURCE MONITOR training_monitor
  CREDIT_QUOTA = 100          -- max 100 credits total
  FREQUENCY = MONTHLY         -- reset quota monthly
  START_TIMESTAMP = IMMEDIATELY
  TRIGGERS
    ON 75 PERCENT DO NOTIFY         -- alert at 75%
    ON 90 PERCENT DO NOTIFY         -- alert at 90%
    ON 100 PERCENT DO SUSPEND;      -- suspend all WH at 100%
  -- ON 110 PERCENT DO SUSPEND_IMMEDIATE; -- force-suspend at 110%

-- Assign monitor to a warehouse:
ALTER WAREHOUSE training_wh SET RESOURCE_MONITOR = training_monitor;

-- View resource monitors:
SHOW RESOURCE MONITORS;

-- View credit usage:
SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.RESOURCE_MONITORS;

-- Who can create resource monitors?
-- Only ACCOUNTADMIN role!
```

TIP Resource Monitors are your budget guardrail. Set them up for every project and every team. A runaway query or forgotten warehouse can burn through credits fast.

Questions — Session 4

Q1	What is a micro-partition? What size are they and how are they created?
Q2	What is query pruning and how does Snowflake achieve it?
Q3	What is a clustering key? When should you add one to a table?
Q4	Explain the 3 types of caching in Snowflake.
Q5	Which cache type is completely free (uses 0 compute credits)?
Q6	What is data spilling? How can you detect it and fix it?
Q7	What does 'Partitions Scanned' vs 'Partitions Total' tell you in Query Profile?
Q8	What is a Resource Monitor and what role is required to create one?
Q9	If a query runs fast the second time but slow the first, which cache is responsible?
Q10	What metrics indicate poor clustering? (overlap, depth)

SESSION 5 — Security & Access Control (3:00–3:40)

5.1 Role-Based Access Control (RBAC)

Snowflake uses Role-Based Access Control (RBAC). Instead of granting permissions directly to users, you grant permissions to ROLES, and then assign roles to users. This makes managing permissions much easier at scale.

Key concept: A user can have multiple roles. The currently active role determines what they can do.

Built-in System Roles

Role	Who Uses It	What It Can Do
ORGADMIN	Organization admin	Manage multiple Snowflake accounts, view organization-level usage
ACCOUNTADMIN	Account administrator	Highest privilege. Manages all objects, billing, users, resource monitors. Use sparingly!
SECURITYADMIN	Security admin	Create/manage users and roles. Cannot see data.
USERADMIN	User admin	Create users and roles. Subset of SECURITYADMIN.
SYSADMIN	System admin	Create databases, warehouses, schemas, tables. Typical DBA role.
PUBLIC	All users (default)	Minimal privileges. All users have this role automatically.

> ⚠ **IMPORTANT** ACCOUNTADMIN is extremely powerful. Best practice: do NOT use ACCOUNTADMIN for daily work. Create a custom role for each team and use SYSADMIN for object creation.

Role Hierarchy & Creating Custom Roles

```
-- Best practice role structure:
-- ACCOUNTADMIN
--    └── SYSADMIN
--          └── DATA_ENGINEER_ROLE
--                └── ANALYST_ROLE

-- Step 1: Create custom roles (as SECURITYADMIN or USERADMIN)
USE ROLE SECURITYADMIN;
CREATE ROLE data_engineer_role COMMENT='Data engineering team';
CREATE ROLE analyst_role       COMMENT='Analytics team';

-- Step 2: Grant privileges to roles
USE ROLE SYSADMIN;

-- Grant database access:
GRANT USAGE ON DATABASE training_db TO ROLE data_engineer_role;
GRANT USAGE ON SCHEMA training_db.raw TO ROLE data_engineer_role;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA training_db.raw TO ROLE data_engi-
neer_role;
GRANT ALL PRIVILEGES ON FUTURE TABLES IN SCHEMA training_db.raw TO ROLE data_engi-
neer_role;
```

```
-- Analysts: read-only on analytics schema
GRANT USAGE ON DATABASE training_db TO ROLE analyst_role;
GRANT USAGE ON SCHEMA training_db.analytics TO ROLE analyst_role;
GRANT SELECT ON ALL VIEWS IN SCHEMA training_db.analytics TO ROLE analyst_role;
GRANT SELECT ON FUTURE TABLES IN SCHEMA training_db.analytics TO ROLE ana-
lyst_role;

-- Step 3: Grant warehouse access:
GRANT USAGE ON WAREHOUSE training_wh TO ROLE data_engineer_role;
GRANT USAGE ON WAREHOUSE training_wh TO ROLE analyst_role;

-- Step 4: Assign roles to users
USE ROLE SECURITYADMIN;
GRANT ROLE data_engineer_role TO USER alice;
GRANT ROLE analyst_role       TO USER bob;

-- Set default role for a user:
ALTER USER alice SET DEFAULT_ROLE = data_engineer_role;

-- Step 5: Role hierarchy (grant role to role):
GRANT ROLE analyst_role TO ROLE data_engineer_role;
-- Now data engineers can switch to analyst_role too

-- Switch roles in a session:
USE ROLE analyst_role;

-- View current role:
SELECT CURRENT_ROLE();
```

Database Roles (Snowflake 2023+)

Database Roles are scoped to a specific database — useful for data sharing and more granular control.

```
-- Create a database role:
CREATE DATABASE ROLE training_db.reader_role;

-- Grant privileges to database role:
GRANT USAGE ON SCHEMA training_db.analytics TO DATABASE ROLE train-
ing_db.reader_role;
GRANT SELECT ON ALL TABLES IN SCHEMA training_db.analytics TO DATABASE ROLE train-
ing_db.reader_role;

-- Grant database role to an account role:
GRANT DATABASE ROLE training_db.reader_role TO ROLE analyst_role;
```

5.2 Common Access Privileges

Privilege	Object	What It Allows
USAGE	Database, Schema, Warehouse	Access the object (required before any other privilege)
SELECT	Table, View	Read data

INSERT	Table	Add new rows
UPDATE	Table	Modify existing rows
DELETE	Table	Remove rows
TRUNCATE	Table	Remove all rows without Time Travel
CREATE TABLE	Schema	Create new tables in the schema
ALL PRIVILEGES	Any	Grants all applicable privileges
OPERATE	Warehouse	Start, stop, resize the warehouse
MONITOR	Warehouse, Database	View usage and query history

5.3 Dynamic Data Masking

Dynamic Data Masking (DDM) lets you hide or obfuscate sensitive data from unauthorized users at query time — without modifying the actual stored data. The data remains unchanged; only the displayed result is masked.

Real-world use: A customer service agent should see the last 4 digits of a credit card. A data analyst should see it fully masked. A senior analyst with special role sees the full number.

```
-- Step 1: Create a masking policy
CREATE OR REPLACE MASKING POLICY email_mask
  AS (val STRING) RETURNS STRING ->
  CASE
    WHEN CURRENT_ROLE() IN ('DATA_ENGINEER_ROLE') THEN val          -- see full
email
    WHEN CURRENT_ROLE() = 'ANALYST_ROLE'          THEN '***@***.com' -- masked
    ELSE '***REDACTED***'                                            -- others:
fully hidden
  END;

-- Step 2: Apply masking policy to a column
ALTER TABLE customers MODIFY COLUMN email
  SET MASKING POLICY email_mask;

-- Test: As engineer (full access):
USE ROLE data_engineer_role;
SELECT email FROM customers;  -- amit@email.com (real)

-- Test: As analyst (masked):
USE ROLE analyst_role;
SELECT email FROM customers;  -- ***@***.com

-- View masking policies:
SHOW MASKING POLICIES;

-- Remove masking policy from column:
ALTER TABLE customers MODIFY COLUMN email UNSET MASKING POLICY;

-- Phone number masking example (show last 4 digits):
```

```
CREATE OR REPLACE MASKING POLICY phone_mask
  AS (val STRING) RETURNS STRING ->
  CASE
    WHEN CURRENT_ROLE() = 'DATA_ENGINEER_ROLE' THEN val
    ELSE REGEXP_REPLACE(val, '.(?=.{4})', '*')
  END;
```

FREE TIER Dynamic Data Masking is available on Enterprise Edition and above. On Standard/Free trial, you can create policies but they may not enforce properly — check Snowflake docs for current feature availability by edition.

5.4 Row Access Policy (Row-Level Security)

Row Access Policy controls WHICH ROWS a user can see in a table, based on their role, attributes, or other conditions. Different users querying the same table see different rows automatically.

Example: Each regional manager can only see orders from their own region.

```
-- Create a mapping table (who can see which region):
CREATE TABLE region_access_map (
  user_role VARCHAR,
  region    VARCHAR
);

INSERT INTO region_access_map VALUES
  ('ANALYST_NORTH', 'North'),
  ('ANALYST_SOUTH', 'South'),
  ('DATA_ENGINEER_ROLE', 'North'),  -- engineers see all
  ('DATA_ENGINEER_ROLE', 'South');

-- Create the Row Access Policy:
CREATE OR REPLACE ROW ACCESS POLICY region_policy
  AS (region_col VARCHAR) RETURNS BOOLEAN ->
  CASE
    WHEN CURRENT_ROLE() = 'ACCOUNTADMIN' THEN TRUE  -- admins see everything
    WHEN EXISTS (
      SELECT 1 FROM region_access_map
      WHERE user_role = CURRENT_ROLE()
        AND region = region_col
    ) THEN TRUE
    ELSE FALSE
  END;

-- Apply the policy to the orders table:
ALTER TABLE orders ADD ROW ACCESS POLICY region_policy ON (region);

-- Users with ANALYST_NORTH role now automatically get only North region rows
-- The WHERE clause is transparent — they don't know they're filtered

-- View row access policies:
SHOW ROW ACCESS POLICIES;

-- Remove policy:
ALTER TABLE orders DROP ROW ACCESS POLICY region_policy;
```

5.5 External Tokenization

External Tokenization replaces sensitive values (e.g., credit card numbers) with tokens using an EXTERNAL tokenization service (like Protegrity, Voltage, etc.). Unlike masking (which shows masked data), tokenization replaces the value with a meaningless token that can be de-tokenized by authorized services.

```
-- External tokenization requires an external function connected to a tokenization
service
-- Concept flow:
-- 1. Raw data: '4111-1111-1111-1111' (credit card)
-- 2. On INSERT: tokenization service returns 'TKN-8f7a2b1c'
-- 3. Stored in Snowflake: 'TKN-8f7a2b1c'
-- 4. Authorized user with detokenization access sees: '4111-1111-1111-1111'
-- 5. Unauthorized user sees: 'TKN-8f7a2b1c' (meaningless token)

-- Tokenization policy uses EXTERNAL FUNCTION (connects to external API):
CREATE OR REPLACE MASKING POLICY tokenized_card_policy
  AS (val STRING) RETURNS STRING ->
  CASE
    WHEN CURRENT_ROLE() IN ('PCI_ADMIN') THEN detokenize_card(val)  -- external
function
    ELSE val  -- others see the token (already stored as token)
  END;
```

> ⃠ **FREE TIER** External Tokenization requires Business Critical Edition and a connection to an external tokenization vendor. Not available on Standard/Free tier.

Questions — Session 5

Q1	What is RBAC and how does it work in Snowflake?
Q2	List all built-in Snowflake roles and their responsibilities.
Q3	What is the difference between SECURITYADMIN and SYSADMIN?
Q4	What privilege is required before any other privilege can be used on a database?
Q5	What is Dynamic Data Masking and how does it differ from encrypting data?
Q6	What is a Row Access Policy? Give a real-world use case.
Q7	How do you apply a masking policy to a column?
Q8	What is the difference between Dynamic Data Masking and External Tokenization?
Q9	What is a Database Role and how is it different from an Account Role?
Q10	What does GRANT SELECT ON FUTURE TABLES IN SCHEMA ... do?

SESSION 6 — Data Sharing, Replication & Cost Optimization

6.1 Secure Data Sharing — Provider & Consumer

Snowflake's Data Sharing lets you share LIVE data with other Snowflake accounts without copying it. The consumer sees your data in real time — any changes you make are immediately visible to them.

Key facts:

- NO data movement — the consumer reads directly from the provider's storage
- Consumer pays for compute (their warehouse), provider pays for storage
- Can share: Tables, External Tables, Secure Views, Secure Materialized Views, UDFs
- Consumer must also be a Snowflake account

```
-- ============================
-- PROVIDER SIDE: Share data
-- ============================

-- Step 1: Create a share object
CREATE SHARE sales_share COMMENT='Share Q1 sales data';

-- Step 2: Grant database and schema access to the share
GRANT USAGE ON DATABASE training_db TO SHARE sales_share;
GRANT USAGE ON SCHEMA training_db.analytics TO SHARE sales_share;

-- Step 3: Grant access to specific objects
GRANT SELECT ON TABLE training_db.analytics.orders TO SHARE sales_share;
-- Or share a secure view (recommended — hides your data logic):
GRANT SELECT ON VIEW training_db.analytics.v_customer_orders TO SHARE sales_share;

-- Step 4: Add consumer accounts to the share
ALTER SHARE sales_share ADD ACCOUNTS = 'consumer_account_locator';

-- View shares you've created:
SHOW SHARES;

-- ============================
-- CONSUMER SIDE: Access shared data
-- ============================

-- Step 1: Create a database from the share
CREATE DATABASE shared_sales_db FROM SHARE provider_account.sales_share;

-- Step 2: Query the shared data (read-only):
SELECT * FROM shared_sales_db.analytics.v_customer_orders;
-- Data is live — any changes provider makes are instantly visible
```

6.2 Data Exchange & Snowflake Marketplace

A Data Exchange is a private hub where an organization can share data with specific partners or customers. Think of it as a private version of the public Snowflake Marketplace.

- Snowflake Marketplace: Public marketplace with 2,000+ datasets from 350+ providers (weather, financial, demographic, etc.)
- Private Exchange: Your own branded data exchange for sharing with specific partners

```
-- Access Snowflake Marketplace:
-- Snowsight → Data → Marketplace
-- Browse free and paid datasets

-- Example: Get free weather data from Weather Source:
-- 1. Find 'Weather Source LLC: frostbyte' on Marketplace
-- 2. Click 'Get' → choose database name
-- 3. Query it immediately:
SELECT date_valid_std, avg_temperature_air_2m_f
FROM weathersource.standard_tile.history_day
WHERE postal_code = '10001'
  AND date_valid_std >= '2024-01-01'
ORDER BY date_valid_std;
```

NOTE The Snowflake Marketplace is available on all editions including the free trial. You can instantly access many free datasets — try exploring it!

6.3 Replication — Multi-Region & Multi-Cloud

Replication lets you replicate a Snowflake database from one account/region to another for disaster recovery, geo-redundancy, or multi-cloud strategy.

```
-- Enable replication on a database (primary account):
ALTER DATABASE training_db ENABLE REPLICATION TO ACCOUNTS aws_eu_west_1.my_eu_ac-
count;

-- On the SECONDARY account: create a replica
CREATE DATABASE training_db_replica
  AS REPLICA OF primary_account.training_db;

-- Refresh the replica (sync latest data):
ALTER DATABASE training_db_replica REFRESH;

-- Failover: promote secondary to primary
ALTER DATABASE training_db_replica PRIMARY;

-- View replication status:
SHOW REPLICATION DATABASES;
```

FREE TIER Replication requires Business Critical Edition or higher and separate account setup. Cannot be tested on a single free trial account.

6.4 Snowflake Scripting

Snowflake Scripting extends SQL with procedural capabilities — variables, IF/ELSE, loops, exception handling. It is the language used inside stored procedures and can also be used in worksheets.

```
-- Basic Snowflake Script (run in a worksheet with BEGIN...END block):
DECLARE
  v_total_orders INT;
  v_message      VARCHAR;
BEGIN
```

```
  -- Assign a variable
  SELECT COUNT(*) INTO :v_total_orders FROM orders;

  -- IF/ELSEIF/ELSE:
  IF (v_total_orders > 100) THEN
    v_message := 'High volume: ' || v_total_orders || ' orders';
  ELSEIF (v_total_orders > 50) THEN
    v_message := 'Medium volume: ' || v_total_orders || ' orders';
  ELSE
    v_message := 'Low volume: ' || v_total_orders || ' orders';
  END IF;

  RETURN v_message;
END;

-- FOR loop:
DECLARE
  total INT DEFAULT 0;
BEGIN
  FOR i IN 1 TO 10 DO
    total := total + i;
  END FOR;
  RETURN total;  -- returns 55
END;

-- Exception handling:
BEGIN
  INSERT INTO orders VALUES (NULL, NULL, NULL, NULL, NULL);
EXCEPTION
  WHEN STATEMENT_ERROR THEN
    RETURN 'Insert failed: ' || SQLERRM;
END;

-- WHILE loop with cursor:
DECLARE
  c1 CURSOR FOR SELECT order_id FROM orders WHERE status='PENDING';
  v_id INT;
BEGIN
  OPEN c1;
  LOOP
    FETCH c1 INTO v_id;
    IF (v_id IS NULL) THEN BREAK; END IF;
    UPDATE orders SET status='PROCESSED' WHERE order_id = v_id;
  END LOOP;
  CLOSE c1;
  RETURN 'Done processing pending orders';
END;
```

6.5 Cost Optimization — Best Practices

Snowflake bills on two dimensions: Storage ($/TB/month) and Compute ($/credit/second). Here are the key strategies to optimize costs:

Compute Cost Optimization

- Always set AUTO_SUSPEND — default to 60-120 seconds for interactive, longer for batch
- Use the RIGHT size warehouse — don't use XL for small queries
- Use Snowpark Container Services or serverless tasks instead of always-on warehouses
- Query Profile: fix spilling, improve pruning, reduce bytes scanned
- Use RESULT_SCAN to reuse query results instead of re-running queries
- Leverage metadata cache — queries like COUNT(*), MIN/MAX are often free
- Schedule heavy jobs during off-peak hours

Storage Cost Optimization

- Use TRANSIENT tables for staging and intermediate data (no 7-day fail-safe)
- Use TEMPORARY tables for within-session scratch data
- Set appropriate DATA_RETENTION_TIME — don't keep 90 days if you only need 1
- Regularly DROP unused tables, stages, databases
- Use compression: Snowflake auto-compresses, but choose efficient formats (Parquet > CSV for large loads)

Cost Visibility

```
-- View daily compute credit usage by warehouse:
SELECT warehouse_name, DATE_TRUNC('day', start_time) AS usage_day,
       SUM(credits_used) AS daily_credits
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE start_time >= DATEADD(DAY, -30, CURRENT_TIMESTAMP)
GROUP BY 1, 2
ORDER BY 3 DESC;

-- View storage usage by database:
SELECT database_name,
       ROUND(AVERAGE_DATABASE_BYTES/1e9, 2)        AS avg_db_gb,
       ROUND(AVERAGE_FAILSAFE_BYTES/1e9, 2)        AS avg_failsafe_gb,
       ROUND(AVERAGE_STAGE_BYTES/1e9, 2)           AS avg_stage_gb
FROM SNOWFLAKE.ACCOUNT_USAGE.DATABASE_STORAGE_USAGE_HISTORY
WHERE USAGE_DATE = CURRENT_DATE - 1
ORDER BY avg_db_gb DESC;

-- View most expensive queries (last 24h):
SELECT query_id, query_text,
       ROUND(execution_time/1000, 1) AS exec_seconds,
       bytes_scanned/1e9             AS gb_scanned,
       credits_used_cloud_services
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE start_time >= DATEADD(HOUR, -24, CURRENT_TIMESTAMP)
ORDER BY execution_time DESC
LIMIT 20;
```

6.6 Query Acceleration Service (QAS) & Search Optimization Service (SOS)

Query Acceleration Service (QAS)

QAS automatically offloads parts of a query to serverless compute, helping queries that have unpredictable or skewed workloads finish faster without you needing to resize the warehouse.

```
-- Enable QAS on a warehouse:
ALTER WAREHOUSE training_wh SET
  ENABLE_QUERY_ACCELERATION = TRUE
  QUERY_ACCELERATION_MAX_SCALE_FACTOR = 8;
-- Scale factor 8 = QAS can use up to 8x the warehouse size as extra compute

-- Which queries benefit from QAS?
-- Queries with: large scans, high selectivity filters, unpredictable data skew
-- Queries that do NOT benefit: small scans, simple lookups, fully cached results

-- Check if a query was accelerated:
SELECT query_id, query_acceleration_bytes_scanned
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE query_acceleration_bytes_scanned > 0;
```

> **FREE TIER** QAS is available on Enterprise Edition and above. On Standard/Free, the parameter can be set but may not activate acceleration.

Search Optimization Service (SOS)

SOS dramatically speeds up selective point lookups and substring searches on large tables by building an optimized search index in the background.

```
-- Enable Search Optimization on a table:
ALTER TABLE customers ADD SEARCH OPTIMIZATION;

-- Check status (indexing happens in background):
SHOW TABLES LIKE 'customers';  -- see SEARCH_OPTIMIZATION_PROGRESS column

-- SOS is especially fast for queries like:
SELECT * FROM customers WHERE email = 'amit@email.com';
-- Or partial matches:
SELECT * FROM customers WHERE email LIKE '%@gmail.com';
-- Or JSON field lookups:
SELECT * FROM customer_events WHERE event_raw:user_id::INT = 101;

-- SOS on specific columns (more targeted, cheaper):
ALTER TABLE customers ADD SEARCH OPTIMIZATION ON EQUALITY(email), SUBSTRING(cus-
tomer_name);

-- Remove search optimization:
ALTER TABLE customers DROP SEARCH OPTIMIZATION;
```

> **FREE TIER** Search Optimization Service requires Enterprise Edition. It also has ongoing storage costs for the search index (approximately 1x to 4x your table's compressed size). Use only for tables where selective lookups are frequent and slow.

Final Comprehensive Lab

NOTE This lab covers: Streams, Tasks, UDFs, Semi-structured data, Masking, and Cost views. All parts marked FREE work on free tier.

```
-- ============================================
-- PART 1: Semi-Structured Data Lab (FREE)
-- ============================================
USE DATABASE training_db;
USE SCHEMA raw;
USE WAREHOUSE training_wh;

CREATE OR REPLACE TABLE product_catalog (
  product_id INT,
  details    VARIANT
);

INSERT INTO product_catalog SELECT 1,
  PARSE_JSON('{"name":"Laptop","brand":"Dell","specs":{"ram":"16GB","stor-
age":"512GB SSD"},
  "tags":["electronics","computing","premium"],"price":75000,"in_stock":true}');

INSERT INTO product_catalog SELECT 2,
  PARSE_JSON('{"name":"Headphones","brand":"Sony","specs":{"type":"Over-
ear","wireless":true},
  "tags":["audio","electronics"],"price":12000,"in_stock":true}');

INSERT INTO product_catalog SELECT 3,
  PARSE_JSON('{"name":"Keyboard","brand":"Logitech","specs":{"type":"Mechani-
cal","wireless":false},
  "tags":["peripherals","computing"],"price":4500,"in_stock":false}');

-- Query with dot notation:
SELECT product_id,
       details:name::VARCHAR     AS product_name,
       details:brand::VARCHAR    AS brand,
       details:specs.ram::VARCHAR AS ram,
       details:price::FLOAT      AS price,
       details:in_stock::BOOLEAN AS in_stock
FROM product_catalog;

-- FLATTEN tags array:
SELECT p.product_id, p.details:name::VARCHAR AS product, f.value::VARCHAR AS tag
FROM product_catalog p,
     LATERAL FLATTEN(INPUT => p.details:tags) f
ORDER BY product_id, tag;

-- ============================================
-- PART 2: UDF Lab (FREE)
-- ============================================
CREATE OR REPLACE FUNCTION price_category(price FLOAT)
RETURNS VARCHAR
LANGUAGE SQL AS
```

```
$$
  CASE
    WHEN price < 5000  THEN 'Budget'
    WHEN price < 20000 THEN 'Mid-Range'
    ELSE 'Premium'
  END
$$;

SELECT details:name::VARCHAR AS product,
       details:price::FLOAT  AS price,
       price_category(details:price::FLOAT) AS category
FROM product_catalog;

-- ================================================
-- PART 3: Stream + Task Lab (FREE)
-- ================================================
CREATE OR REPLACE STREAM product_changes_stream ON TABLE product_catalog;

-- Log table to capture changes:
CREATE OR REPLACE TABLE product_audit_log (
  log_time    TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP,
  product_id  INT,
  action      VARCHAR,
  product_name VARCHAR
);

-- Task to process stream every minute:
CREATE OR REPLACE TASK log_product_changes
  WAREHOUSE = training_wh
  SCHEDULE = '1 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('product_changes_stream')
AS
INSERT INTO product_audit_log (product_id, action, product_name)
SELECT product_id,
       METADATA$ACTION,
       details:name::VARCHAR
FROM product_changes_stream;

ALTER TASK log_product_changes RESUME;

-- Trigger a change:
UPDATE product_catalog SET details = OBJECT_INSERT(details,'price',5000,TRUE)
WHERE product_id = 3;

-- Check stream:
SELECT product_id, METADATA$ACTION, details:name::VARCHAR FROM prod-
uct_changes_stream;

-- Manually run task to test:
EXECUTE TASK log_product_changes;

-- Check audit log:
```

```
SELECT * FROM product_audit_log;

ALTER TASK log_product_changes SUSPEND;

-- ================================================
-- PART 4: Access Control Lab (FREE)
-- ================================================
USE ROLE SECURITYADMIN;
CREATE ROLE IF NOT EXISTS lab_analyst;
GRANT USAGE ON DATABASE training_db TO ROLE lab_analyst;
GRANT USAGE ON SCHEMA training_db.raw TO ROLE lab_analyst;
GRANT SELECT ON TABLE training_db.raw.product_catalog TO ROLE lab_analyst;
GRANT USAGE ON WAREHOUSE training_wh TO ROLE lab_analyst;

-- Create masking policy:
USE ROLE ACCOUNTADMIN;
CREATE OR REPLACE MASKING POLICY price_mask
  AS (val FLOAT) RETURNS FLOAT ->
  CASE
    WHEN CURRENT_ROLE() = 'SYSADMIN' THEN val
    ELSE -1  -- masked price
  END;

ALTER TABLE training_db.raw.product_catalog
  MODIFY COLUMN details SET MASKING POLICY price_mask;
-- NOTE: Masking VARIANT columns has limitations; apply to extracted view for best
results

-- ================================================
-- PART 5: Cost Visibility (FREE)
-- ================================================
-- View warehouse usage:
SELECT warehouse_name, SUM(credits_used) AS total_credits
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE start_time >= DATEADD(DAY,-7,CURRENT_TIMESTAMP)
GROUP BY 1 ORDER BY 2 DESC;

-- View your most expensive queries today:
SELECT query_text,
       ROUND(execution_time/1000,1) AS seconds,
       bytes_scanned/1e6 AS mb_scanned
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE start_time >= DATEADD(DAY,-1,CURRENT_TIMESTAMP)
  AND execution_status='SUCCESS'
ORDER BY execution_time DESC LIMIT 10;
```

🎯 Final Session Questions — Data Sharing & Cost

Q1	What is Secure Data Sharing? Does data get copied to the consumer's account?
Q2	Who pays for compute in Secure Data Sharing — provider or consumer?
Q3	What is the difference between Snowflake Marketplace and a Private Data Exchange?

Q4	What is database replication used for in Snowflake?
Q5	What is the Query Acceleration Service (QAS) and what types of queries does it help?
Q6	What is the Search Optimization Service (SOS) and when should you use it?
Q7	Name at least 5 compute cost optimization techniques in Snowflake.
Q8	Which ACCOUNT_USAGE view shows daily credit usage by warehouse?
Q9	What is Snowflake Scripting and how is it different from regular SQL?
Q10	A table has millions of rows and a selective lookup query is very slow. What service can help?

Complete Summary — Key Concepts

Topic	Key Takeaway
Bulk vs Continuous Loading	COPY INTO = manual/batch. Snowpipe = event-driven, automatic, serverless.
Snowpipe	Triggered by cloud events. Create pipe with AUTO_INGEST=TRUE. Get SQS ARN from SHOW PIPES.
Streams	CDC object. Tracks INSERT/UPDATE/DELETE. METADATA$ACTION and METADATA$ISUPDATE are key columns.
Tasks	Scheduled SQL. ROOT task has SCHEDULE. Child tasks have AFTER clause. Resume children first, root last.
DAG of Tasks	Chain tasks with AFTER clause. Build ETL pipelines inside Snowflake.
Stream + Task	WHEN SYSTEM$STREAM_HAS_DATA() prevents wasted compute. Most powerful automation pattern.
UDF	Custom function returning 1 value. Supports SQL, JS, Python, Java, Scala.
Table UDF (UDTF)	Custom function returning multiple rows. Used with TABLE() syntax.
Stored Procedure	Multi-step logic. Can commit/rollback. Called with CALL. EXECUTE AS CALLER or OWNER.
Sequences	Auto-increment IDs. Not gap-free. Use .NEXTVAL in INSERT.
Sampling	BERNOULLI=row-level. SYSTEM=block-level. Fixed ROWS for exact count.
Semi-Structured	VARIANT stores JSON. Access with : (colon) + dot/bracket. Cast with :: (double colon).
FLATTEN	Explodes arrays into rows. LATERAL FLATTEN(INPUT => col:array_field) f.
Unstructured Data	3 URL types: Scoped (expiring, authed), File (permanent, authed), Pre-signed (public, expiring).
Micro-partitioning	Auto 50-500MB compressed files. Snowflake manages them. Columnar storage.

Query Pruning	Skips partitions based on min/max metadata. Better with clustering or naturally sorted data.
Clustering Key	Re-organizes data for better pruning. ALTER TABLE t CLUSTER BY (col). Has background maintenance cost.
Result Cache	24h, free, returns identical query results instantly.
Metadata Cache	Free. Answers COUNT(*), MIN, MAX from metadata without scanning data.
Warehouse Cache	SSD cache on nodes. Lost on WH suspend. Speeds up repeated table scans.
Data Spilling	Memory overflow → SSD → Remote storage. Fix: scale up WH or optimize query.
Resource Monitor	Budget guardrail. Set CREDIT_QUOTA. Can NOTIFY or SUSPEND. Only ACCOUNTADMIN creates.
RBAC	Roles → Privileges → Objects. Users assigned roles. SECURITYADMIN creates roles. SYSADMIN creates objects.
Dynamic Data Masking	Masks column values at query time based on role. Data not changed. Enterprise+.
Row Access Policy	Controls which rows a user sees. Transparent filtering. Enterprise+.
Data Sharing	Live, zero-copy sharing between Snowflake accounts. Consumer sees real-time data.
Snowflake Scripting	Variables, IF/ELSE, loops, cursors, exception handling. Used in stored procedures and scripts.
Cost Optimization	AUTO_SUSPEND, right-size WH, use transient tables, fix spilling, use caches, Resource Monitors.

